

Sección 6: Patrones que Conectan las Piezas

- **Adapter:** conecta algo externo que no encaja (API de fraude → interfaz del dominio).
- **Facade:** conecta varios subsistemas detrás de una operación simple (dashboard).
- **Chain of Responsibility:** conecta validadores independientes en secuencia.
- **Proxy:** conecta un intermediario invisible entre el cliente y el bean real.
- **State:** conecta el comportamiento correcto según el estado actual del objeto.
- **Builder:** conecta los pasos de construcción en una API fluida.

La diferencia no es perfecta al 100%. State tiene algo de "organizar lógica" y Observer tiene algo de "conectar piezas". Pero como criterio general para que el alumno entienda por qué hay dos secciones de patrones y no una sola, funciona bien. La Sección 5 mira hacia adentro del service, la Sección 6 mira hacia los bordes y las conexiones entre componentes.

68. Refactorizando: enums

The image displays six code snippets from an IDE, illustrating the refactoring of enums and their usage in Java classes.

AccountType.java and **AccountStatus.java** (top left):

```
package com.atlas.bank.atlas.account.model;

public enum AccountType {
    SAVINGS,
    CHECKING,
    PREMIUM
}
```

```
package com.atlas.bank.atlas.account.model;

public enum AccountStatus {
    ACTIVE,
    FROZEN,
    CLOSED
}
```

TransactionStatus.java and **TransactionType.java** (top right):

```
package com.atlas.bank.atlas.transaction.model;

public enum TransactionStatus {
    EXECUTED
}
```

```
package com.atlas.bank.atlas.transaction.model;

public enum TransactionType {
    DEPOSIT,
    WITHDRAWAL,
    TRANSFER
}
```

Account.java (bottom left):

```
public class Account {
    @EqualsAndHashCode.Include
    private Long id;

    @Column(name = "account_number", nullable = false, unique = true)
    private String accountNumber;

    @Column(name = "owner_name", nullable = false)
    private String ownerName;

    @Column(nullable = false)
    private String email;

    @Enumerated(EnumType.STRING)
    @Column(nullable = false, length = 20)
    private AccountType type; // SAVING, CHECKING

    @Column(nullable = false)
    private BigDecimal balance;

    @Enumerated(EnumType.STRING)
    @Column(nullable = false, length = 20)
    private AccountStatus status; // ACTIVE, CLOSED, FROZEN

    @Column(name = "created_at", nullable = false, updatable = false)
    private LocalDateTime createdAt;

    @PrePersist
    public void prePersist() {
        this.createdAt = LocalDateTime.now();
        if (this.status == null) this.status = AccountStatus.ACTIVE;
        if (this.balance == null) this.balance = BigDecimal.ZERO;
    }
}
```

Transaction.java (bottom right):

```
public class Transaction {
    @Id
    private Long id;

    @Column(nullable = false, length = 20)
    @Enumerated(EnumType.STRING)
    private TransactionType type; // DEPOSIT, WITHDRAWAL, TRANSFER

    @Column(name = "source_account_id")
    private Long sourceAccountId;

    @Column(name = "target_account_id")
    private Long targetAccountId;

    @Column(nullable = false)
    private BigDecimal amount;

    @Column(nullable = false)
    private BigDecimal fee;

    @Enumerated(EnumType.STRING)
    private TransactionStatus status; // PENDING, EXECUTED, REJECTED

    @Column(name = "created_at", nullable = false, updatable = false)
    private LocalDateTime createdAt;

    @PrePersist
    public void prePersist() {
        this.createdAt = LocalDateTime.now();
        if (this.status == null) this.status = TransactionStatus.EXECUTED;
    }
}
```

➔Modificando clases:

```
FeeCalculator.java × TransactionFactory.java CreateAccountRequest.java DefaultFeeCalculator.java
1 package com.atlas.bank.atlas.transaction.service.fee;
2
3 import com.atlas.bank.atlas.account.model.AccountType;
4
5 import java.math.BigDecimal;
6
7 public interface FeeCalculator { 7 usages 4 implementations ydochoadev *
8     boolean supports(AccountType accountType); 1 usage 4 implementations new *
9
10    BigDecimal calculate(BigDecimal amount); 1 usage 4 implementations ydochoadev
11 }
12
```

```
FeeCalculator.java TransactionFactory.java × CreateAccountRequest.java DefaultFeeCalculator.java
1 package com.atlas.bank.atlas.transaction.service.factory;
2
3 > import ...
9
10 public class TransactionFactory { 2 usages ydochoadev *
11
12     public static Transaction createTransfer(TransferContext context, BigDecimal fee) {
13         Transaction transaction = new Transaction();
14         transaction.setType(TransactionType.TRANSFER);
15         transaction.setSourceAccountId(context.from().getId());
16         transaction.setTargetAccountId(context.to().getId());
17         transaction.setAmount(context.amount());
18         transaction.setFee(fee);
19         transaction.setStatus(TransactionStatus.EXECUTED);
20
21         return transaction;
22     }
23 }
```

```
FeeCalculator.java TransactionFactory.java CreateAccountRequest.java ×
1 package com.atlas.bank.atlas.account.dto;
2
3 import com.atlas.bank.atlas.account.model.AccountType;
4 import jakarta.validation.constraints.Email;
5 import jakarta.validation.constraints.NotBlank;
6 import jakarta.validation.constraints.NotNull;
7 import jakarta.validation.constraints.PositiveOrZero;
8 import lombok.Data;
9
10 import java.math.BigDecimal;
11
12 // @Data => Genera getter, setter, toString, equalsAndHashCode,
13 // problema en las entity que tienen relaciones: Data usado en D
14 @Data 5 usages ydochoadev *
15 public class CreateAccountRequest {
16     @NotBlank(message = "El número de cuenta es obligatorio")
17     private String accountNumber;
18
19     @NotBlank(message = "El nombre del titular es obligatorio")
20     private String ownerName;
21
22     @NotBlank(message = "El correo es obligatorio")
23     @Email(message = "El correo no tiene un formato válido")
24     private String email;
25
26     @NotNull(message = "El tipo de cuenta es obligatorio")
27     private AccountType type;
28
29     @PositiveOrZero(message = "El saldo no puede set negativo")
30     private BigDecimal balance;
31 }
32
```

```
FeeCalculator.java TransactionFactory.java CreateAccountRequest.java DefaultFeeCalculator.java
1 package com.atlas.bank.atlas.transaction.service.fee;
2
3 import com.atlas.bank.atlas.account.model.AccountType;
4 import org.springframework.core.annotation.Order;
5 import org.springframework.stereotype.Component;
6
7 import java.math.BigDecimal;
8
9 // @Order => Por defecto, se coloca el final de una lista
10 @Component @ydochoadev *
11 @Order()
12 public class DefaultFeeCalculator implements FeeCalculator {
13     @Override @ydochoadev 1 usage
14     public BigDecimal calculate(BigDecimal amount) { return BigDecimal.ZERO; }
15
16
17     @Override @ydochoadev 1 usage
18     public boolean supports(AccountType accountType) { return true; }
19
20
21 }
22
```

```
AtlasApplication
FeeCalculator.java TransactionFactory.java CreateAccountRequest.java DefaultFeeCalculator.java CheckingFeeCalculator.java
1 package com.atlas.bank.atlas.transaction.service.fee;
2
3 import com.atlas.bank.atlas.account.model.AccountType;
4 import org.springframework.core.annotation.Order;
5 import org.springframework.stereotype.Component;
6
7 import java.math.BigDecimal;
8
9 @Component @ydochoadev *
10 @Order(1)
11 public class CheckingFeeCalculator implements FeeCalculator {
12     @Override @ydochoadev 1 usage
13     public BigDecimal calculate(BigDecimal amount) { return amount.multiply(new BigDecimal("0.015")); }
14
15
16     @Override @ydochoadev 1 usage new *
17     public boolean supports(AccountType accountType) { return accountType == AccountType.CHECKING; }
18
19
20 }
21
```

```
FeeCalculator.java TransactionFactory.java CreateAccountRequest.java DefaultFeeCalculator.java
1 package com.atlas.bank.atlas.transaction.service.fee;
2
3 import com.atlas.bank.atlas.account.model.AccountType;
4 import org.springframework.core.annotation.Order;
5 import org.springframework.stereotype.Component;
6
7 import java.math.BigDecimal;
8
9 @Component @ydochoadev *
10 @Order(1)
11 public class PremiumFeeCalculator implements FeeCalculator {
12     @Override @ydochoadev 1 usage
13     public BigDecimal calculate(BigDecimal amount) { return BigDecimal.ZERO; }
14
15
16     @Override @ydochoadev 1 usage new *
17     public boolean supports(AccountType accountType) {
18         return accountType != AccountType.PREMIUM;
19     }
20
21 }
```

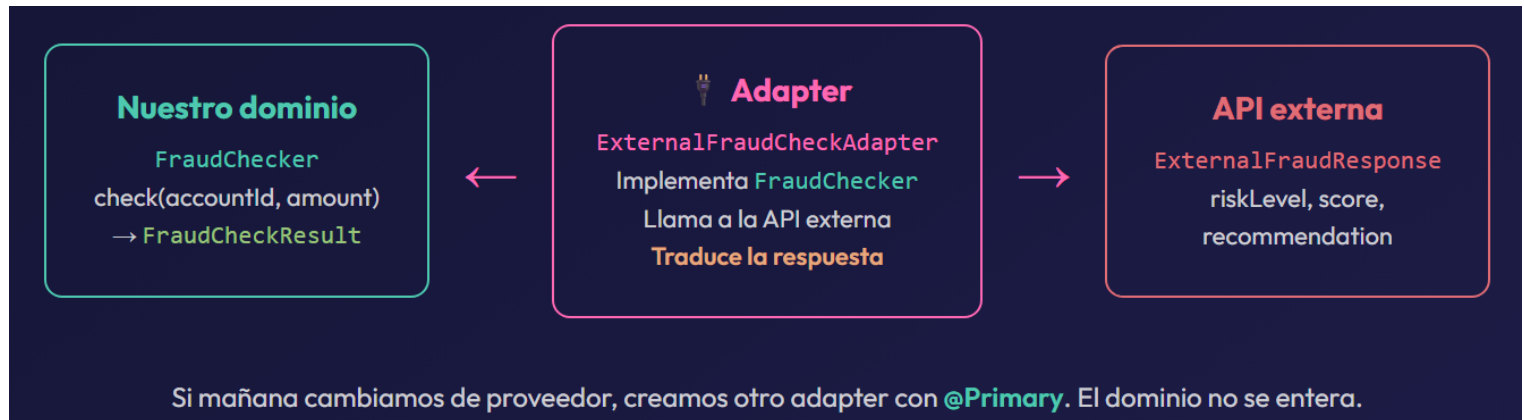
```
ator.java TransactionFactory.java CreateAccountRequest.java DefaultFeeCalculator.java CheckingFeeCalculator.java Premium
1 package com.atlas.bank.atlas.transaction.service.fee;
2
3 import com.atlas.bank.atlas.account.model.AccountType;
4 import org.springframework.core.annotation.Order;
5 import org.springframework.stereotype.Component;
6
7 import java.math.BigDecimal;
8
9 @Component @ydochoadev *
10 @Order(1)
11 public class SavingsFeeCalculator implements FeeCalculator {
12     @Override @ydochoadev 1 usage
13     public BigDecimal calculate(BigDecimal amount) { return amount.multiply(new BigDecimal("0.01")); }
14
15
16     @Override @ydochoadev 1 usage new *
17     public boolean supports(AccountType accountType) {
18         return accountType == AccountType.PREMIUM;
19     }
20
21 }
22
```

69. Adapter: integrar lo que no encaja - parte 1

→ El problema:

- Necesitamos verificar **fraude** antes de ejecutar una transferencia
- El servicio externo devuelve: **riskLevel, score, recommendation**
- Nuestro dominio solo necesita saber:
¿Está bloqueada? ¿Por qué?
- Dos mundos que **no hablan el mismo idioma**

→ El Adapter traduce entre los dos mundos



70. Adapter: integrar lo que no encaja - parte 2

→ Representación de la respuesta de la API externa

```
ExternalFraudResponse.java
1 package com.atlas.bank.atlas.transaction.service.fraud;
2
3 import lombok.AllArgsConstructor;
4 import lombok.Getter;
5
6 @Getter no usages
7 @AllArgsConstructor
8 public class ExternalFraudResponse {
9     private String riskLevel; // LOW, MEDIUM, HIGH
10    private double score; // 0.0, 1.0
11    private String recommendation; // ALLOW, REVIEW, BLOCK
12 }
```

→ Definir lo que el dominio necesita: El TransferService solo necesita preguntar si está permitido o si está bloqueado.

```
ExternalFraudResponse.java  FraudCheckResult.java
1 package com.atlas.bank.atlas.transaction.service.fraud;
2
3 public record FraudCheckResult(boolean blocked, String reason) { 4 usages
4
5     @ public static FraudCheckResult allowed() { no usages
6         return new FraudCheckResult( blocked: false, reason: null);
7     }
8
9     @ public static FraudCheckResult blocked(String reason) { no usages
10        return new FraudCheckResult( blocked: true, reason);
11    }
12 }
13 }
```

→ Interface que nuestro servicio va a usar:

```
ExternalFraudResponse.java  FraudCheckResult.java  FraudChecker.java x
1  package com.atlas.bank.atlas.transaction.service.fraud;
2
3  import java.math.BigDecimal;
4
5  public interface FraudChecker { no usages
6      ⚡ FraudCheckResult check(Long accountId, BigDecimal amount);
7  }
```

→ Adapter: Clase que traduce entre los 2 mundos.

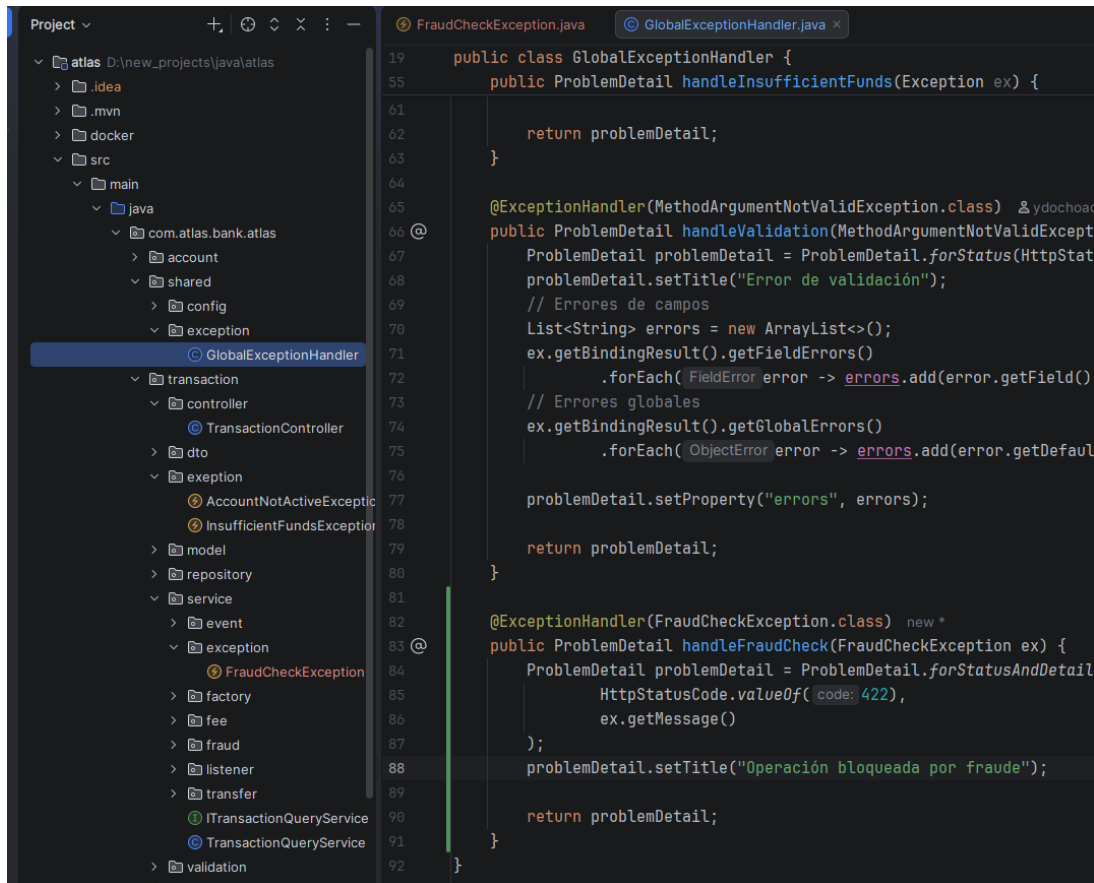
```
ExternalFraudResponse.java  ExternalFraudCheckAdapter.java x  FraudCheckResult.java  FraudChecker.java
1  package com.atlas.bank.atlas.transaction.service.fraud;
2
3  import lombok.extern.slf4j.Slf4j;
4  import org.springframework.stereotype.Component;
5
6  import java.math.BigDecimal;
7
8  @Component
9  @Slf4j
10 public class ExternalFraudCheckAdapter implements FraudChecker {
11     @Override no usages
12     public FraudCheckResult check(Long accountId, BigDecimal amount) {
13         ExternalFraudResponse fraudResponse = callExternalApi(accountId, amount);
14         log.info("Respuesta del servicio de fraude - riesgo: {}, score: {}, recomendación: {}",
15             fraudResponse.getRiskLevel(), fraudResponse.getScore(), fraudResponse.getRecommendation());
16         if ("BLOCK".equals(fraudResponse.getRiskLevel())) {
17             return FraudCheckResult.blocked(reason: "Operación bloqueada por riesgo " + fraudResponse.getRiskLevel()
18                 + " (score: " + fraudResponse.getScore() + ")");
19         }
20         return FraudCheckResult.allowed();
21     }
22
23     private ExternalFraudResponse callExternalApi(Long accountId, BigDecimal amount) { 1 usage
24         if (amount.compareTo(new BigDecimal("1000000")) > 0) {
25             return new ExternalFraudResponse(riskLevel: "HIGH", score: 0.95, recommendation: "BLOCK");
26         }
27
28         return new ExternalFraudResponse(riskLevel: "LOW", score: 0.1, recommendation: "ALLOW");
29     }
30 }
```

71. Adapter: integrar lo que no encaja - parte 3

→ Crear una excepción específica para la API de fraude:

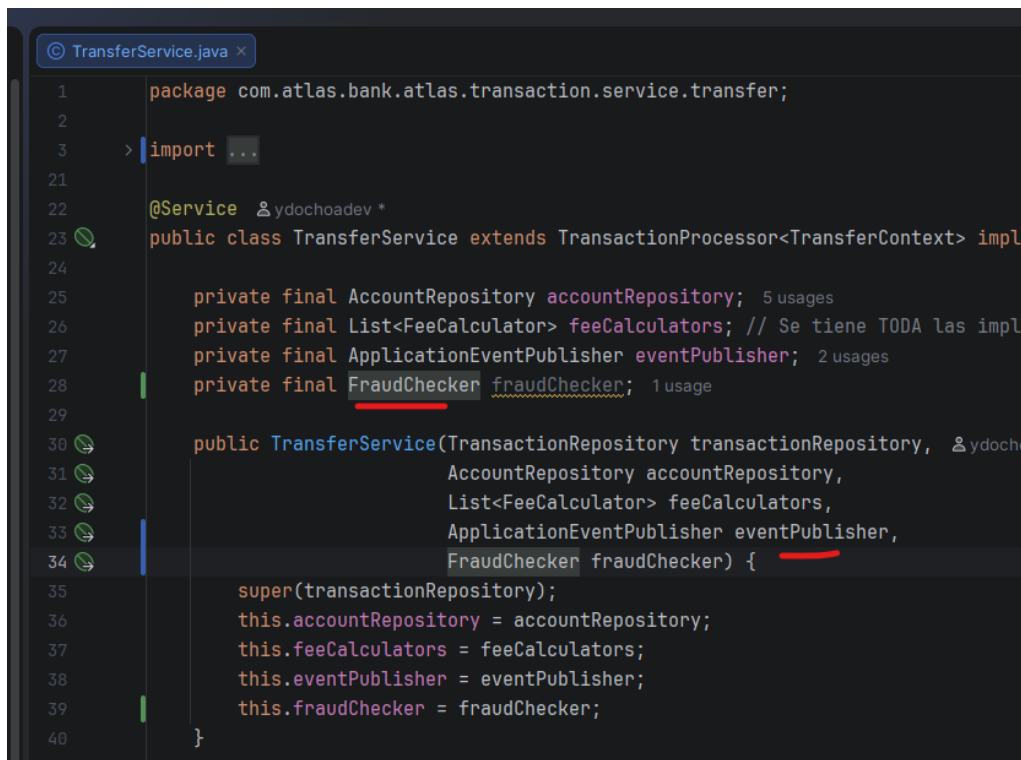
```
ExternalFraudResponse.java  ExternalFraudCheckAdapter.java  FraudCheckException.java x
1  package com.atlas.bank.atlas.transaction.service.exception;
2
3  public class FraudCheckException extends RuntimeException { no usages
4      > public FraudCheckException(String reason) { super(reason); }
7  }
```

→ Colocarla en el mapeo de excepciones:



```
19 public class GlobalExceptionHandler {
55     public ProblemDetail handleInsufficientFunds(Exception ex) {
61         return problemDetail;
62     }
63 }
64
65 @ExceptionHandler(MethodArgumentNotValidException.class) & ydochoad
66 @
67 public ProblemDetail handleValidation(MethodArgumentNotValidExcept
68     ProblemDetail problemDetail = ProblemDetail.forStatus(HttpStat
69     problemDetail.setTitle("Error de validación");
70     // Errores de campos
71     List<String> errors = new ArrayList<>();
72     ex.getBindingResult().getFieldErrors()
73         .forEach( FieldError error -> errors.add(error.getField()
74     // Errores globales
75     ex.getBindingResult().getGlobalErrors()
76         .forEach( ObjectError error -> errors.add(error.getDefault
77
78     problemDetail.setProperty("errors", errors);
79
80     return problemDetail;
81 }
82
83 @ExceptionHandler(FraudCheckException.class) new *
84 @
85 public ProblemDetail handleFraudCheck(FraudCheckException ex) {
86     ProblemDetail problemDetail = ProblemDetail.forStatusAndDetail
87     HttpStatusCode.valueOf( code: 422),
88     ex.getMessage()
89 );
90 problemDetail.setTitle("Operación bloqueada por fraude");
91
92     return problemDetail;
93 }
94 }
```

→ Integrar:



```
1 package com.atlas.bank.atlas.transaction.service.transfer;
2
3 > import ...
4
51
52 @Service & ydochoad
53 @
54 public class TransferService extends TransactionProcessor<TransferContext> impl
55
56
57     private final AccountRepository accountRepository; 5 usages
58     private final List<FeeCalculator> feeCalculators; // Se tiene TODA las impl
59     private final ApplicationEventPublisher eventPublisher; 2 usages
60     private final FraudChecker fraudChecker; 1 usage
61
62
63     public TransferService(TransactionRepository transactionRepository, & ydocho
64         AccountRepository accountRepository,
65         List<FeeCalculator> feeCalculators,
66         ApplicationEventPublisher eventPublisher,
67         FraudChecker fraudChecker) {
68         super(transactionRepository);
69         this.accountRepository = accountRepository;
70         this.feeCalculators = feeCalculators;
71         this.eventPublisher = eventPublisher;
72         this.fraudChecker = fraudChecker;
73     }
74 }
```

```
TransferService.java x
25 public class TransferService extends TransactionProcessor<TransferContext> implements ITransferService {
66
67     @Override 1 usage ydochoadev *
68     @protected void validate(TransferContext context) {
69         // Validar que la cuenta esté activa
70         if (context.from().getStatus() != AccountStatus.ACTIVE) {
71             throw new AccountNotActiveException(context.from().getId(), context.from().getStatus().name());
72         }
73         if (context.to().getStatus() != AccountStatus.ACTIVE) {
74             throw new AccountNotActiveException(context.to().getId(), context.to().getStatus().name());
75         }
76
77         // Validar fondos
78         if (context.from().getBalance().compareTo(context.amount()) < 0) {
79             throw new InsufficientFundsException(context.from().getId(), context.from().getBalance(), context.amount());
80         }
81
82         FraudCheckResult fraudCheckResult = fraudChecker.check(context.from().getId(), context.amount());
83         if (fraudCheckResult.blocked()) {
84             throw new FraudCheckException(fraudCheckResult.reason());
85         }
86     }
}
```

72. Facade: simplificar subsistemas complejos 1

➔ El problema: Cuando un cliente necesita hablar con varios subsistemas para armar una respuesta. Evitamos que el controller conozca a todos esos subsistemas, se crea una clase que los coordina y expone una sola operación simple.

NO TIENE LÓGICA DE NEGOCIO, solo orquesta llamadas y arma respuestas.

✗ SIN FACADE — Controller coordina todo

```
AccountController.java
Account account = accountService.findById(id);
List<Transaction> txns =
    queryService.getByAccountId(id);
FraudCheckResult fraud =
    fraudChecker.check(id, balance);
// Armar el DTO combinando todo...
// 3 dependencias, 3 llamadas
```

✓ CON FACADE — Una sola llamada

```
AccountController.java
DashboardResponse dashboard =
    facade.getDashboard(id);
// 1 dependencia, 1 llamada
// La coordinación vive en el Facade
```



→ Creando DTO de respuesta en account:

```
1 package com.atlas.bank.atlas.account.dto;
2
3 import com.atlas.bank.atlas.transaction.dto.TransactionResponse;
4 import lombok.Builder;
5 import lombok.Data;
6
7 import java.math.BigDecimal;
8 import java.util.List;
9
10 @Data 5 usages ydochoadev *
11 @Builder
12 public class DashboardResponse {
13     private Long accountId;
14     private String accountNumber;
15     private String ownerName;
16     private String type;
17     private BigDecimal balance;
18     private String status;
19     List<TransactionResponse> recentTransactions;
20 }
```

→ Creando nuevo servicio para coordinar los 3 subsistemas: Account, Transaction y Fraud.

```
1 package com.atlas.bank.atlas.account.service;
2
3 import com.atlas.bank.atlas.account.dto.DashboardResponse;
4 import com.atlas.bank.atlas.account.model.Account;
5 import com.atlas.bank.atlas.transaction.dto.TransactionMapper;
6 import com.atlas.bank.atlas.transaction.dto.TransactionResponse;
7 import com.atlas.bank.atlas.transaction.service.ITransactionQueryService;
8 import lombok.RequiredArgsConstructor;
9 import org.springframework.stereotype.Service;
10
11 import java.util.List;
12
13 @Service 2 usages ydochoadev *
14 @RequiredArgsConstructor
15 public class AccountDashboardFacade {
16     private final IAccountService accountService;
17     private final ITransactionQueryService transactionQueryService;
18     private final TransactionMapper transactionMapper;
19
20     public DashboardResponse getDashboard(Long accountId) { 1 usage ydochoadev
21         Account account = this.accountService.findById(accountId);
22         // Lista de trx
23         List<TransactionResponse> transactions = transactionQueryService
24             .getByAccountId(accountId) List<Transaction>
25             .stream() Stream<Transaction>
26             .map(transactionMapper::toResponse) Stream<TransactionResponse>
27             .toList();
28
29         return DashboardResponse.builder()
30             .accountId(account.getId())
31             .accountNumber(account.getAccountNumber())
32             .ownerName(account.getOwnerName())
33             .type(account.getType().name())
34             .balance(account.getBalance())
35             .status(account.getStatus().name())
36             .recentTransactions(transactions)
37             .build();
38     }
39 }
```

73. Facade: simplificar subsistemas complejos 2

→Controller:

```
AccountDashboardFacade.java | AccountController.java x
1 package com.atlas.bank.atlas.account.controller;
2
3 > import ...
22
23 @RestController @ydochoadev *
24 @RequestMapping("/api/v1/accounts")
25 @AllArgsConstructor
26 public class AccountController {
27
28     private final IAccountService accountService;
29     private final AccountMapper accountMapper;
30     private final AccountDashboardFacade dashboardFacade;
31
32     @GetMapping("/{id}/dashboard") new *
33     public ResponseEntity<DashboardResponse> getDashboard(@PathVariable Long id) {
34         return ResponseEntity.ok(dashboardFacade.getDashboard(id));
35     }
36
37     @PostMapping @ydochoadev
38     public ResponseEntity<AccountResponse> create(@Valid @RequestBody CreateAccountRequest
39         Account account = this.accountMapper.toEntity(request);
40
41         Account accountCreated = accountService.create(account);
42         return ResponseEntity.status(HttpStatus.CREATED).body(this.accountMapper.toResponse(accountCreated));
43     }
44 }
```

74. Chain of responsability: Procesamiento en cadena

→El problema

✗ HOY — Un bloque que crece

TransferService.java

```
void validate(TransferContext ctx) {
    if (cuenta origen no activa) throw...
    if (cuenta destino no activa) throw...
    if (fondos insuficientes) throw...
    if (fraude bloqueado) throw...
    // ¿Límite diario? Otro if
    // ¿Compliance? Otro if
}
```

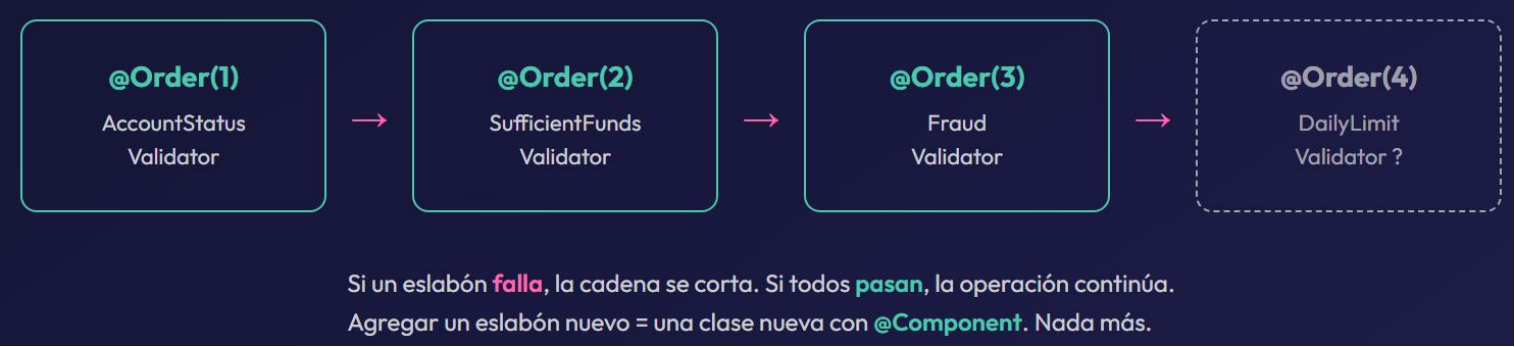
✓ CON CHAIN — Validadores independientes

TransferService.java

```
void validate(TransferContext ctx) {
    validators.forEach(v →
        v.validate(ctx));
}

// Agregar validación =
// nueva clase + @Component
// Cero cambios en TransferService
```

→Solución: Cadena de validación



Lo que se busca es solucionar este método de validación:

```
@Override 1 usage ydochoadev
protected void validate(TransferContext context) {
    // Validar que la cuenta esté activa
    if (context.from().getStatus() != AccountStatus.ACTIVE) {
        throw new AccountNotActiveException(context.from().getId(), context.from().getStatus().name());
    }
    if (context.to().getStatus() != AccountStatus.ACTIVE) {
        throw new AccountNotActiveException(context.to().getId(), context.to().getStatus().name());
    }

    // Validar fondos
    if (context.from().getBalance().compareTo(context.amount()) < 0) {
        throw new InsufficientFundsException(context.from().getId(), context.from().getBalance(), context.amount());
    }

    FraudCheckResult fraudCheckResult = fraudChecker.check(context.from().getId(), context.amount());
    if (fraudCheckResult.blocked()) {
        throw new FraudCheckException(fraudCheckResult.reason());
    }
}
```

→ Creando interfaz

```
TransferService.java  TransferValidator.java x
1 package com.atlas.bank.atlas.transaction.validation.chain;
2
3 import com.atlas.bank.atlas.transaction.service.transfer.TransferContext;
4
5 public interface TransferValidator { no usages
6     void validate(TransferContext context); no usages
7 }
8
```

→ Implementando primer eslabón, para Account:

```
TransferService.java  TransferValidator.java  StatusAccountValidator.java x
1 package com.atlas.bank.atlas.transaction.validation.chain;
2
3 import com.atlas.bank.atlas.account.model.AccountStatus;
4 import com.atlas.bank.atlas.transaction.exception.AccountNotActiveException;
5 import com.atlas.bank.atlas.transaction.service.transfer.TransferContext;
6 import org.springframework.core.annotation.Order;
7 import org.springframework.stereotype.Component;
8
9 @Component
10 @Order(1)
11 public class StatusAccountValidator implements TransferValidator {
12     @Override no usages
13     public void validate(TransferContext context) {
14         if (context.from().getStatus() != AccountStatus.ACTIVE) {
15             throw new AccountNotActiveException(context.from().getId(), context.from().getStatus().name());
16         }
17         if (context.to().getStatus() != AccountStatus.ACTIVE) {
18             throw new AccountNotActiveException(context.to().getId(), context.to().getStatus().name());
19         }
20     }
21 }
```

➔ Implementando segundo eslabón, para fondos:

```
© TransferService.java  © TransferValidator.java  © StatusAccountValidator.java  © SuffucentFundsValidator.java x
1  package com.atlas.bank.atlas.transaction.validation.chain;
2
3  import com.atlas.bank.atlas.transaction.expection.InsufficientFundsException;
4  import com.atlas.bank.atlas.transaction.service.transfer.TransferContext;
5  import org.springframework.core.annotation.Order;
6  import org.springframework.stereotype.Component;
7
8  @Component
9  @Order(2)
10 public class SuffucentFundsValidator implements TransferValidator {
11     @Override no usages
12     public void validate(TransferContext context) {
13         if (context.from().getBalance().compareTo(context.amount()) < 0) {
14             throw new InsufficientFundsException(context.from().getId(), context.from().getBalance(), context.amount());
15         }
16     }
17 }
```

➔ Implementando tercer eslabón, para fondos:

```
© TransferService.java  © FraudValidator.java x  © TransferValidator.java  © StatusAccountValidator.java  © SuffucentFundsValidator.java
1  package com.atlas.bank.atlas.transaction.validation.chain;
2
3  import com.atlas.bank.atlas.transaction.service.exception.FraudCheckException;
4  import com.atlas.bank.atlas.transaction.service.fraud.FraudCheckResult;
5  import com.atlas.bank.atlas.transaction.service.fraud.FraudChecker;
6  import com.atlas.bank.atlas.transaction.service.transfer.TransferContext;
7  import lombok.RequiredArgsConstructor;
8  import org.springframework.core.annotation.Order;
9  import org.springframework.stereotype.Component;
10
11 @Component
12 @Order(3)
13 @RequiredArgsConstructor
14 public class FraudValidator implements TransferValidator {
15     private final FraudChecker fraudChecker;
16
17     @Override no usages
18     public void validate(TransferContext context) {
19         FraudCheckResult fraudCheckResult = fraudChecker.check(context.from().getId(), context.amount());
20         if (fraudCheckResult.blocked()) {
21             throw new FraudCheckException(fraudCheckResult.reason());
22         }
23     }
24 }
```

➔ Modificando el servicio transfer:

```
© TransferService.java x  © FraudValidator.java  © TransferValidator.java  © StatusAccountValidator.java  © SuffucentFundsValidator.java
1  package com.atlas.bank.atlas.transaction.service.transfer;
2
3  > import ...
18
19 @Service @ydochoadev *
20 public class TransferService extends TransactionProcessor<TransferContext> implements ITransferService {
21
22     private final AccountRepository accountRepository; 5 usages
23     private final List<FeeCalculator> feeCalculators; // Se tiene TODA las implementaciones 2 usages
24     private final ApplicationEventPublisher eventPublisher; 2 usages
25     private final List<TransferValidator> validators; 2 usages
26
27     public TransferService(TransactionRepository transactionRepository, @ydochoadev *
28         AccountRepository accountRepository,
29         List<FeeCalculator> feeCalculators,
30         ApplicationEventPublisher eventPublisher,
31         List<TransferValidator> validators) {
32         super(transactionRepository);
33         this.accountRepository = accountRepository;
34         this.feeCalculators = feeCalculators;
35         this.eventPublisher = eventPublisher;
36         this.validators = validators;
37     }
}
```

```
@Override 1 usage ydochoadev *
protected void validate(TransferContext context) {
    // Validar que la cuenta esté activa. Fondos. Y fraude
    validators.forEach( TransferValidator validator -> validator.validate(context));
}
```

75. Proxy: control de acceso transparente

→ Decorator vs Proxy



→ Spring genera Proxies por nosotros.

- **@Cacheable** → verifica el caché, si el dato está **no delega**
- **@Transactional** → abre transacción, delega, y hace commit o rollback
- **@Async** → ejecuta el método en otro thread
- Todos generados automáticamente con **CGLIB** en runtime
- Nosotros vamos a implementar uno a mano para entender el mecanismo

→ Se quita anotación Primary en AuditableAccountService, ya que habrá otro servicio que será usado como proxy.

```

AuditableAccountService.java x
1 package com.atlas.bank.atlas.account.service;
2
3 > import ...
10
11 @Slf4j ydochoadev
12 @Component
13 @Primary ✗
14 public class AuditableAccountService implements IAccountService {

```

➔ Creando servicio proxy:

Controller => CachedAccountService => AuditableAccountService => AccountService (servicio real)

```
© AuditableAccountService.java  © CachedAccountService.java x
1  package com.atlas.bank.atlas.account.service;
2
3  > import ...
12
13  @Slf4j
14  @Component
15  @Primary
16  public class CachedAccountService implements IAccountService {
17      private final IAccountService delegate; 4 usages
18      private final Map<Long, Account> cache = new ConcurrentHashMap<>(); 3 usages
19
20      public CachedAccountService(@Qualifier("auditableAccountService") IAccountService delegate) {
21          this.delegate = delegate;
22      }
23
24      @Override
25      public Account create(Account account) {
26          Account created = delegate.create(account);
27          cache.put(created.getId(), created);
28          log.info("Cuenta {} agregada al caché", created.getId());
29          return created;
30      }
31
32      @Override 3 usages
33      public List<Account> findAll() {
34          return delegate.findAll();
35      }
36
37      @Override 4 usages
38      public Account findById(Long id) {
39          Account cached = cache.get(id);
40          if (cached != null) {
41              log.info("Cuenta {} encontrada en caché - No se llama al servicio real", id);
42              return cached;
43          }
44          log.info("Cuenta {} no encontrada en caché - Delegando al servicio real ", id);
45          Account account = delegate.findById(id);
46          cache.put(id, account);
47
48          return account;
```

76. Proxy en Spring: de manual a @Cacheable

➔ Agregar dependencia para usar el caché:

```
m pom.xml (atlas) x
2  <project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://
34  <dependencies> Add Starters...
36      <groupId>org.springframework.boot</groupId>
37      <artifactId>spring-boot-h2console</artifactId>
38  </dependency>
39  <!-- cache -->
40  <dependency>
41      <groupId>org.springframework.boot</groupId>
42      <artifactId>spring-boot-starter-cache</artifactId>
43  </dependency>
44  <dependency>
45      <groupId>org.springframework.boot</groupId>
```

➔ Eliminar `CachedAccountService` para usar el cachable de Spring y agregar el `@Primary` al `AuditableAccountService`:

@EnableCaching => Activa el mecanismo de caché en proxys.

```

1 package com.atlas.bank.atlas;
2
3 > import ...
4
5
6
7 @SpringBootApplication
8 @EnableCaching
9 public class AtlasApplication {
10
11     public static void main(String[] args) { SpringApplication.run(AtlasApplication.class, args); }
12
13 }
14
15

```

➔ Colocar en caché las cuentas:

```

1 package com.atlas.bank.atlas.account.service;
2
3 > import ...
4
5 @Service
6 @RequiredArgsConstructor
7 public class AccountService implements IAccountService {
8
9     private final AccountRepository accountRepository;
10
11     @Override
12     @Transactional
13     public Account create(Account account) { return accountRepository.save(account); }
14
15     @Override
16     @Transactional(readOnly = true)
17     public List<Account> findAll() { return accountRepository.findAll(); }
18
19     @Override
20     @Transactional(readOnly = true)
21     @Cacheable(value = "accounts", key = "#id")
22     public Account findById(Long id) {
23         return accountRepository.findById(id).orElseThrow(() -> new AccountNotFoundException(id));
24     }
25 }

```

```

: Initialized org.hibernate.engine.jdbc.connections.internal.DriverManagerConnectionProviderImpl:
: Hibernate is in classpath; If applicable, HQL parser will be used.
: Clase real del accountService: com.atlas.bank.atlas_bank.account.service.AccountService$$$SpringCGLIB$$0
: spring.jpa.open-in-view is enabled by default. Therefore, database queries may be performed during view rendering.

```

**** CGLIB => Librería que spring usa para crear clases en tiempo de ejecución.**

77. State: comportamiento que cambia con el estado 1

→ El problema:

✗ SIN STATE — Switch en cada operación

```
void validate(Transaction tx) {  
    switch (tx.getStatus()) {  
        case PENDING → // ok  
        case EXECUTED → throw...  
        case REJECTED → throw...  
    }  
}  
  
// Repetir switch en execute()  
// Repetir switch en reject()  
// Repetir switch en reverse()
```

✓ CON STATE — El estado decide

```
// Cada estado sabe qué puede hacer  
tx.getState().validate();  
// → PENDING: ok, avanza  
// → EXECUTED: IllegalStateException  
  
// Sin switches, sin ifs  
// Agregar estado = nueva clase
```

→ Ciclo de vida de una Transaction

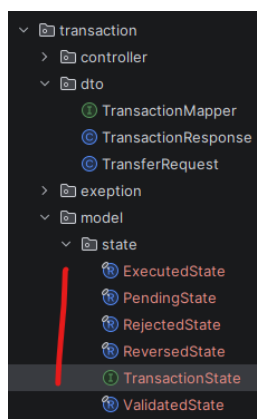


** sealed (Sellado): Solo permitir los accesos que están dentro de su configuración.

→ En transaction, agregar los estados:

```
TransactionStatus.java  
1 package com.atlas.bank.atlas.transaction.model;  
2  
3 public enum TransactionStatus { 4 usages ydoct  
4     PENDING, no usages  
5     VALIDATED, no usages  
6     EXECUTED, 2 usages  
7     REJECTED, no usages  
8     REVERSED no usages  
9 }
```

→ Luego, crear los objetos que representen cada uno de estos estados.



```
PendingState.java x ValidatedState.java ExecutedState.java RejectedState
1 package com.atlas.bank.atlas.transaction.model.state;
2
3 import com.atlas.bank.atlas.transaction.model.TransactionStatus;
4
5 public record PendingState() implements TransactionState { 1 usage
6     @Override
7     public TransactionStatus status() {
8         return TransactionStatus.PENDING;
9     }
10 }
```

```
PendingState.java x ValidatedState.java x ExecutedState.java RejectedState
1 package com.atlas.bank.atlas.transaction.model.state;
2
3 import com.atlas.bank.atlas.transaction.model.TransactionStatus;
4
5 public record ValidatedState() implements TransactionState { 1 us
6     @Override
7     public TransactionStatus status() {
8         return TransactionStatus.VALIDATED;
9     }
10 }
```

```
TransactionState.java x PendingState.java ValidatedState.java ExecutedState.java RejectedState.java ReversedState.java
1 package com.atlas.bank.atlas.transaction.model.state;
2
3 import com.atlas.bank.atlas.transaction.model.TransactionStatus;
4
5 public sealed interface TransactionState permits 9 usages 5 implementations
6     PendingState,
7     ValidatedState,
8     ExecutedState,
9     RejectedState,
10    ReversedState {
11    TransactionStatus status(); 5 implementations
12
13    default TransactionState validate() { no usages
14        throw new IllegalStateException("No se puede ejecutar una transacción en estado " + status());
15    }
16
17    default TransactionState execute() { no usages
18        throw new IllegalStateException("No se puede ejecutar una transacción en estado " + status());
19    }
20
21    Change signature
22    default TransactionState reject(String reason) { no usages
23        throw new IllegalStateException("No se puede ejecutar una transacción en estado " + status());
24    }
25
26    default TransactionState reverse() { no usages
27        throw new IllegalStateException("No se puede ejecutar una transacción en estado " + status());
28    }
29 }
```

78. State: comportamiento que cambia con el estado 2

➔ PENDING puede hacer 2 cosas: validar (llama a **validate**) o rechazar (llama a **reject**). NO ejecuta y revertir.

➔ Implementar esos 2 métodos en PENDING:

** Si PendingState intenta un execute o reverse, lanza la excepción por defecto.

```
TransactionState.java PendingState.java x ValidatedState.java ExecutedState.java
1 package com.atlas.bank.atlas.transaction.model.state;
2
3 import com.atlas.bank.atlas.transaction.model.TransactionStatus;
4
5 public record PendingState() implements TransactionState { 1 usage
6     @Override
7     public TransactionStatus status() {
8         return TransactionStatus.PENDING;
9     }
10
11     @Override no usages new *
12     public TransactionState validate() {
13         return new ValidatedState();
14     }
15
16     @Override no usages new *
17     public TransactionState reverse() {
18         return new RejectedState();
19     }
20 }
```

➔ **VALIDATED** => Puede ser ejecutada o rechazada:

```

1 package com.atlas.bank.atlas.transaction.model.state;
2
3 import com.atlas.bank.atlas.transaction.model.TransactionStatus;
4
5 public record ValidatedState() implements TransactionState {
6     @Override @SuppressWarnings("unused")
7     public TransactionStatus status() {
8         return TransactionStatus.VALIDATED;
9     }
10
11     @Override @SuppressWarnings("unused")
12     public TransactionState execute() {
13         return new ExecutedState();
14     }
15
16     @Override @SuppressWarnings("unused")
17     public TransactionState reject(String reason) {
18         return new RejectedState();
19     }
20 }

```

➔ EXECUTED => Solo puede revertir:

```
TransactionState.java PendingState.java ValidatedState.java ExecutedState.java
1 package com.atlas.bank.atlas.transaction.model.state;
2
3 import com.atlas.bank.atlas.transaction.model.TransactionStatus;
4
5 public record ExecutedState() implements TransactionState { 2 usages
6     @Override @ydochoadev
7     public TransactionStatus status() {
8         return TransactionStatus.EXECUTED;
9     }
10
11     @Override no usages new *
12     public TransactionState reverse() {
13         return new ReversedState();
14     }
15 }
```

➔ REJECTED y REVERSED son estados finales, no sobrescriben ninguna operación.

➔ Agregando el estado en el modelo de la trx.

**** @Transient => Transitorio, campo no se guarda en la BD**

The screenshot shows an IDE with a project tree on the left and a code editor on the right. The project tree shows a hierarchy: `src/main/java/com.atlas.bank.atlas/account`, with `Transaction` selected. The code editor shows the `Transaction` class with the following code:

```

28 public class Transaction {
29
30     @Enumerated(EnumType.STRING)
31     private TransactionStatus status; // PENDING, EXECUTED, REJECTED
32
33     @Column(name = "created_at", nullable = false, updatable = false)
34     private LocalDateTime createdAt;
35
36     @Transient
37     private TransactionState state;
38
39     @PrePersist
40     public void prePersist() {
41         this.createdAt = LocalDateTime.now();
42         if (this.status == null) this.status = TransactionStatus.EXECUTED;
43     }
44
45     public TransactionState getState() {
46         if (state == null) {
47             this.state = switch (this.status) {
48                 case PENDING -> new PendingState();
49                 case VALIDATED -> new ValidatedState();
50                 case EXECUTED -> new ExecutedState();
51                 case REJECTED -> new RejectedState();
52                 case REVERSED -> new ReversedState();
53             };
54         }
55         return state;
56     }
57
58     public void advanceTo(TransactionState newState) {
59         this.state = newState;
60         this.status = newState.status();
61     }
62 }

```

79. State pattern: La integración al servicio

Status: Es una propiedad simple que indica en qué punto del camino está algo. Por ejemplo, el "status" de un pedido en Amazon puede ser Enviado o Entregado.

State: Es la fotografía completa de la memoria de un componente o aplicación. Incluye configuraciones, datos del usuario, si un menú está abierto o cerrado, etc.

→ Cambiando el Factory:

```
TransactionFactory.java x
1 package com.atlas.bank.atlas.transaction.service.factory;
2
3 > import ...
10
11 public class TransactionFactory { 2 usages ydochoadev *
12
13 @ public static Transaction createTransfer(TransferContext context, BigDecimal fee) {
14     Transaction transaction = new Transaction();
15     transaction.setType(TransactionType.TRANSFER);
16     transaction.setSourceAccountId(context.from().getId());
17     transaction.setTargetAccountId(context.to().getId());
18     transaction.setAmount(context.amount());
19     transaction.setFee(fee);
20     transaction.setStatus(TransactionStatus.PENDING);
21     transaction.advanceTo(new PendingState());
22
23     return transaction;
24 }
25 }
```

→ En el servicio, manejar los estados siguientes:

```
TransactionFactory.java  TransferService.java x  TransactionProcessor.java
20 public class TransferService extends TransactionProcessor<TransferContext> implements ITransferService
21 {
22     public TransferService(TransactionRepository transactionRepository, ydochoadev
23 }
24
25 @Override 1 usage ydochoadev *
26 @Transactional
27 public Transaction execute(Long fromId, Long toId, BigDecimal amount) {
28     // Buscar cuentas
29     Account from = accountRepository.findById(fromId)
30         .orElseThrow(() -> new AccountNotFoundException(fromId));
31     Account to = accountRepository.findById(toId)
32         .orElseThrow(() -> new AccountNotFoundException(toId));
33
34     // process => aplica el patrón Template Method
35     Transaction transaction = process(new TransferContext(from, to, amount));
36     // Estados
37     transaction.advanceTo(transaction.getState().validate());
38     transaction.advanceTo(transaction.getState().execute());
39     transactionRepository.save(transaction); // Guarda (actualiza) otra vez la trx con su estado
40
41     // Lanzar el evento
42     eventPublisher.publishEvent(new TransactionExecutedEvent(
43         transaction.getId(),
44         transaction.getType().name(),
45         transaction.getSourceAccountId(),
46         transaction.getTargetAccountId(),
47         transaction.getAmount(),
48         transaction.getFee()
49     ));
50     return transaction;
51 }
52 }
```

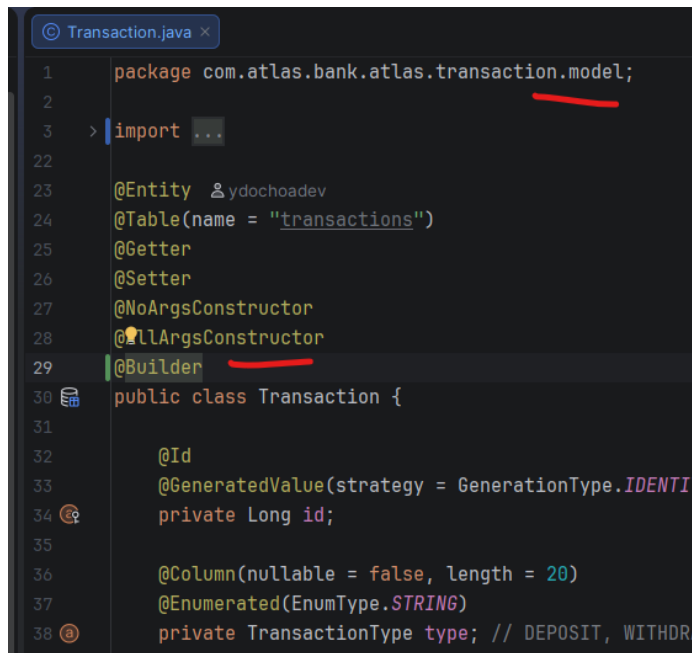
80. Builder: construcción paso a paso

→ El problema



→ Builder no reemplaza a Factory

- **Factory** decide **qué** construir
→ "Una transferencia lleva estos campos con estos valores"
- **Builder** decide **cómo** construirlo
→ "Paso a paso, fluido, sin estados incompletos"
- Con Lombok **@Builder**: una anotación genera todo el mecanismo



Transaction.java

TransactionFactory.java ×

```
1 package com.atlas.bank.atlas.transaction.service.factory;
2
3 > import ...
4
5
6
7
8
9
10
11 public class TransactionFactory { @ydochoadev *
12
13 @ public static Transaction createTransfer(TransferContext context, BigDecimal fee) {
14     Transaction transaction = Transaction.builder()
15         .type(TransactionType.TRANSFER)
16         .sourceAccountId(context.from().getId())
17         .targetAccountId(context.to().getId())
18         .amount(context.amount())
19         .fee(fee)
20         .status(TransactionStatus.PENDING).build();
21
22     transaction.advanceTo(new PendingState());
23
24     return transaction;
25 }
26
27 }
```